

Core REST Principles

Video 3 of "REST API Mastery" series — Easy English study notes to help you understand and explain REST APIs confidently in interviews.

[Backend](#)[MERN](#)[Interview Prep](#)

Statelessness

Statelessness means the server does **not remember** anything about you between requests. Every single request must carry all the information the server needs, every time.

Step 1 — A shopkeeper WHO remembers you: You walk into a shop. The shopkeeper says "Oh hi! Same item as yesterday, right?" He remembers you. You don't need to repeat who you are. This is called **"stateful"** — the shopkeeper is keeping "state" (memory) about you.

Step 2 — A shopkeeper who does NOT remember you: Now imagine a different shopkeeper. Every single time, he asks "Who are you? What do you want?" — even if you visited 5 minutes ago. You must tell him everything, every time, from scratch. This is **"stateless"** — REST APIs work exactly like this shopkeeper.

STATELESS REQUEST — REAL EXAMPLE

Request 1

"Give me user 123's data.
My token: abc123"

Request 2

"Give me user 123's orders.
My token: abc123"

Notice: the token is sent AGAIN in request 2. The server did not remember request 1.

Why this is useful: Since the server doesn't store anything about you, ANY server (out of hundreds) can handle your request. This makes it very easy for companies to scale their APIs.

One line to remember: Stateless = the server has no memory of your past requests. You must introduce yourself, fully, every single time.

How to answer in interview: "Statelessness means each request is independent and must carry all the information the server needs — like the auth token. The server doesn't store any client session data, which makes REST APIs much easier to scale."

The 6 REST Architectural Constraints

REST is not just "use HTTP methods." It is built on 6 official rules (constraints).

- 1 **Client-Server:** Frontend and backend are separate and can change independently.
- 2 **Statelessness:** No client data stored on server between requests.
- 3 **Cacheable:** Responses must say if they can be cached, to improve speed.
- 4 **Uniform Interface:** Consistent way to use URIs, HTTP methods, and data formats.
- 5 **Layered System:** There can be gateways/proxies between client and server, and the client won't even know.
- 6 **Code on Demand (optional):** Server can send executable code to the client, rarely used.

How to answer in interview: "REST has 6 constraints — client-server separation, statelessness, cacheability, uniform interface, layered system, and the optional code-on-demand."

CORS (Cross-Origin Resource Sharing)

CORS is a browser security rule. By default, a browser blocks a website from calling an API hosted on a different domain.

Easy Analogy: Think of CORS like a security guard at an office building. If you (a website) try to enter a different building (a different domain's API), the guard checks a permission list first. If your name isn't on the list, you are not allowed in.

CORS IN ACTION



Server must send "Access-Control-Allow-Origin" header, or the browser blocks the response.

In MERN: When your React frontend (port 3000) calls your Express backend (port 5000), you must enable CORS in Express, since they are different origins.

How to answer in interview: "CORS is a browser security mechanism that controls which domains can call an API. The server allows specific origins using the Access-Control-Allow-Origin header."

CODewithHARDEEP

Liked these notes? **There's more coming.**

This was Video 3 of the "REST API Mastery" series. Next up: Security basics — Authentication, TLS, and how to keep your APIs safe.

- ✓ Subscribe to CodeWithHardeep on YouTube
- ✓ Turn on notifications so you don't miss the next video
- ✓ Comment your doubts — I reply to every comment
- ✓ Watch Video 2 (HTTP Methods & Status Codes) if you missed it

Subscribe to CodeWithHardeep ▶